Custom hooks and rules

16D. Custom Hooks & Rules

Questions: 2 **Difficulty:** Mid to Senior **Companies:** Every company

Q 16.19 How do you build custom hooks? What are the patterns for composition and testing?

Difficulty: Mid **Asked at:** Every company

Custom hooks extract reusable logic from components. They're regular functions that call other hooks — the `use` prefix tells React to enforce hooks rules.

Pattern 1: Encapsulating stateful logic

JSX Copy

```jsx
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const stored = localStorage.getItem(key);
      return stored ? JSON.parse(stored) : initialValue;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);
```

```
// Usage — same API as useState
const [theme, setTheme] = useLocalStorage('theme', 'light');
```

Pattern 2: Composing multiple hooks

JSX Copy

```jsx
function useAsync(asyncFn, deps = []) {
  const [state, setState] = useState({
    data: null,
    error: null,
    loading: true,
  });

  useEffect(() => {
    let cancelled = false;
    setState(prev => ({ ...prev, loading: true, error: null }));

    asyncFn()
      .then(data => {
        if (!cancelled) setState({ data, error: null, loading: false });
      })
      .catch(error => {
        if (!cancelled) setState({ data: null, error, loading: false });
      });

    return () => { cancelled = true; };
  }, deps);

  return state;
}

// Composed into a more specific hook
function useUser(userId) {
  return useAsync(() => fetchUser(userId), [userId]);
}
```

Pattern 3: Event-based hooks

JSX

Copy

```jsx
function useWindowSize() {
  const [size, setSize] = useState({ width: 0, height: 0 });

  useEffect(() => {
    function handleResize() {
      setSize({ width: window.innerWidth, height: window.innerHeight });
    }

    handleResize();  // Set initial value
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return size;
}

function useMediaQuery(query) {
  const [matches, setMatches] = useState(false);

  useEffect(() => {
    const mediaQuery = window.matchMedia(query);
    setMatches(mediaQuery.matches);

    const handler = (e) => setMatches(e.matches);
    mediaQuery.addEventListener('change', handler);
    return () => mediaQuery.removeEventListener('change', handler);
  }, [query]);

  return matches;
}

// Usage
const isMobile = useMediaQuery('(max-width: 768px)');
```

Custom hook design principles:

- Return the minimum needed (don't expose internal state the consumer doesn't need)
- Accept configuration as parameters (make the hook flexible)
- Follow the `use` naming convention (required for hooks rules enforcement)
- Handle cleanup in useEffect returns (prevent leaks)
- Document the hook's API with TypeScript types

Q 16.20 Why do the Rules of Hooks exist? What breaks without them?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart

The two rules: hooks must be called at the top level (not inside conditions, loops, or nested functions), and hooks must be called from React functions (components or custom hooks).

Why top-level only:

React identifies hooks by their call order. On every render, React walks through the hooks in the same order and matches each hook call to its stored state. If you call hooks conditionally, the order changes between renders, and React pairs hooks with the wrong state.

JSX Copy

```jsx
// BROKEN: Hook call order changes between renders
function Profile({ showAge }) {
  const [name, setName] = useState('');       // Hook 1

  if (showAge) {
    const [age, setAge] = useState(0);        // Hook 2 (sometimes)
  }
```

```
}
```

JAVASCRIPT Copy

```
Render 1 (showAge = true):          Render 2 (showAge = false):
  Hook 1 → useState('') ✓             Hook 1 → useState('') ✓
  Hook 2 → useState(0) ✓              Hook 2 → useContext(Theme) ✗ MISMATCH
  Hook 3 → useContext(Theme) ✓          React expected useState(0) here
```

The fix — move the condition inside the hook's usage, not around the hook call:

JSX Copy

```jsx
function Profile({ showAge }) {
  const [name, setName] = useState('');
  const [age, setAge] = useState(0);          // Always called
  const theme = useContext(ThemeContext);

  // Use the value conditionally, not the hook
  return (
    <div>
      <p>{name}</p>
      {showAge && <p>Age: {age}</p>}
    </div>
  );
}
```

Why React functions only:

Hooks need access to React's internal fiber tree to store and retrieve state. Regular JavaScript functions don't have this connection. Calling `useState` outside a component would have nowhere to store the state.

How ESLint catches violations:

JSON Copy

```
  "plugins": ["react-hooks"],
  "rules": {
    "react-hooks/rules-of-hooks": "error",
    "react-hooks/exhaustive-deps": "warn"
  }
}
```

`react-hooks/rules-of-hooks` catches conditional hook calls and hooks outside components. `react-hooks/exhaustive-deps` catches missing dependencies in useEffect/useMemo/useCallback. Both should be enabled on every React project.

The exception: `use()` (Q16.18) can be called conditionally. It's the only hook that breaks this rule, and it was specifically designed for it.

QUICK CHECK — no marks, just for you.

1. A component calls useState unconditionally, then calls a second useState inside an if (showAge) block, then calls useContext. Why does React break when showAge flips from true to false?

- ○ React matches each hook call to its stored state by call order, so skipping a hook shifts the positions and useContext receives the slot that belonged to the conditional useState
- ○ React compares the hook function references between renders, and the conditional useState is a new reference each time
- ○ React requires the number of state variables to match the number of props, and showAge removes one without removing a prop
- ○ React re-runs the effect cleanup for every hook when a condition changes, and the skipped useState never gets cleaned up

Check answer

2. Why can't a custom hook be called from a plain JavaScript helper function that is not a component or a custom hook?

- It works at runtime, but the ESLint rule forbids it purely as a style convention
- Plain functions are not transpiled by the React compiler, so the useState call is stripped out at build time
- Hooks need access to React's internal fiber tree to store and retrieve state, and a regular function has no connection to it, so there would be nowhere to store the state
- useState is bound to the module scope of the file that renders the component, so importing it elsewhere breaks the binding

Check answer

3. According to the lesson, which statement about calling hooks conditionally is accurate?

- There are no exceptions; every hook, including use(), must be called at the top level
- use() is the single exception — it was specifically designed to be callable conditionally, unlike every other hook
- Any hook may be called conditionally as long as it is called the same number of times in every render branch
- useContext and use() may both be called conditionally because neither one stores state in a hook slot

Check answer

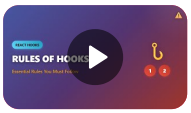
Frontend System Desi...

LEARN MORE

ARTICLES & DOCS

- Rules of Hooks
  react.dev
- Reusing logic with custom hooks
  react.dev

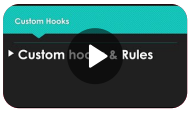
VIDEOS



Rules of hooks for beginners
YouTube



Custom hooks every dev should know
YouTube



Custom hooks and rules
YouTube

Hand-picked references for this lesson.